

CICIDS2017 Dataset Processing Pipeline (Day 4)

Objective

Transform raw CICIDS2017 CSV files into a clean, scaled, machine-learning-ready dataset that exactly matches the feature format produced by the live Flow Engine.

Step 1: Load Raw Dataset Files

Function

`load_all_csvs()`

Process

- Read all CICIDS2017 CSV files from `data/raw/`
- Load each file into a Pandas DataFrame
- Merge all DataFrames into a single dataset

Purpose

Different files contain different attack categories. Combining all files creates a dataset that represents a wide variety of network behaviors and attacks.

Step 2: Clean Column Names

Function

`clean_column_names()`

Process

- Remove leading spaces
- Remove trailing spaces

Example:

" Flow Duration" → "Flow Duration"

Purpose

Ensures consistent and reliable column access throughout the pipeline.

Step 3: Construct the Final Feature Set

Function

build_feature_dataframe()

Input

CICIDS2017 (~79 columns)

Output

13 Features

1. duration
2. packet_count
3. byte_count
4. packets_per_second
5. bytes_per_second
6. avg_packet_size
7. syn_count
8. ack_count
9. fin_count
10. rst_count
11. syn_ratio
12. rst_ratio
13. dst_port

Additional Processing

Binary Label Creation

BENIGN → 0

All Attack Types → 1

Examples:

- DoS
- DDoS
- PortScan
- Bot
- Brute Force
- Web Attack

become:

1 (Attack)

Purpose

Creates a feature set identical to the one generated by the live Flow Engine, ensuring compatibility between training and real-time detection.

Step 4: Data Cleaning

Function

`clean_data()`

Process

Replace Infinite Values

$+\infty$ and $-\infty$

↓

NaN

Remove Missing Values

Drop all rows containing NaN values.

Remove Invalid Flows

Delete rows where:

$\text{duration} \leq 0$

Purpose

Removes:

- Corrupted records
- Invalid measurements
- Mathematically impossible flows

This prevents the model from learning from bad data.

Step 5: Analyze Class Distribution

Function

`show_class_distribution()`

Process

Calculate:

- Number of benign flows
- Number of attack flows
- Attack percentage

Purpose

Verify dataset balance before training.

Example:

Benign: 2,000,000

Attack: 500,000

Attack Rate: 20%

Step 6: Separate Features and Labels

Function

`normalize_and_split()`

Process

Features (X):

duration packet_count byte_count ... dst_port

Labels (y):

0 = Benign

1 = Attack

Purpose

Machine learning requires:

$X \rightarrow$ Inputs

$y \rightarrow$ Outputs

Step 7: Feature Scaling

Tool

StandardScaler

Process

Learn:

`mean(feature)`

`standard_deviation(feature)`

Apply:

`scaled_value = (value - mean) / std`

Result

Every feature becomes standardized with:

Mean ≈ 0

Standard Deviation ≈ 1

Purpose

Prevents large numerical features from dominating the learning process.

Improves model convergence and performance.

Step 8: Train-Test Split

Function

`train_test_split()`

Configuration

Training Set = 80%

Testing Set = 20%

Random State = 42

Stratify = y

Purpose

Training Set: Used for learning patterns.

Testing Set: Used for evaluating model performance on unseen data.

Step 9: Stratification

Configuration

`stratify = y`

Purpose

Preserves class balance across train and test sets.

Example:

Original Dataset:

80% Benign

20% Attack

Training Set:

80% Benign

20% Attack

Testing Set:

80% Benign

20% Attack

This ensures fair evaluation.

Step 10: Save Processed Data

Function

`save_processed()`

Generated Files

`data/processed/`

|— `X_train.npy`

|— `X_test.npy`

|— `y_train.npy`

|— `y_test.npy`

|— `scaler.pkl`

Purpose

Avoid repeating expensive preprocessing operations every time the model is trained.

Step 11: Save Scaler

File

`scaler.pkl`

Contains

- Feature means

- Feature standard deviations
- Scaling parameters

Purpose

Required during live prediction.

Any incoming flow must be scaled using the same scaler that was used during training.

Complete Pipeline Flow

Raw CICIDS2017 CSV Files

↓

Load All Files

↓

Merge Dataset

↓

Clean Column Names

↓

Extract 13 Features

↓

Generate Binary Labels

↓

Clean Invalid Data

↓

Check Class Distribution

↓

Separate X and y



StandardScaler



Scale Features



Train-Test Split (80/20)



Save Processed Data



Save Scaler



Dataset Ready for Model Training

Connection to Live IDS

TRAINING PIPELINE

CICIDS2017



Feature Extraction



Scaling



Model Training

↓

Saved Model

LIVE DETECTION PIPELINE

Network Packets

↓

Flow Engine

↓

Same 13 Features

↓

Load scaler.pkl

↓

scaler.transform()

↓

Trained Model

↓

Prediction

↓

Benign / Attack

Key Principle

The feature calculations used during training must be identical to the feature calculations used by the live Flow Engine.

Training Features = Live Features

If feature definitions differ, the model will perform poorly in real-world deployment even if training accuracy is high.